
Лабораторная работа №0

Техническое задание

Используемые термины и сокращения

АТД Абстрактный тип данных

1. Постановка задачи

На языке C++ реализовать АТД «Последовательность». Написать unit-тесты. Реализовать среду сборки.

2. Функциональные требования

2.1. АТД должен позволять хранить элементы любых типов. Для этого класс должен быть объявлен как шаблонный. Пример:

```
template<typename TElement> class Sequence {
public:
    int GetLength();
    ...
    <TElement> GetFirst();
    <TElement> GetLast();
    <TElement> Get(int index);
    Sequence<TElement> GetSubsequence(int i, int j);
    ...
}
```

TElement – это параметр, обозначающий тип элемента

2.2. АТД «Последовательность» («Sequence») должен обладать следующими атрибутами и методами:

2.3. Атрибуты

	Название	Сигнатура	Назначение
2.3.1.	Length	int getLength()	Длина последовательности (количество элементов)
2.3.2.	IsEmpty	int getIsEmpty()	Признак того, является ли последовательность пустой

2.4. Методы (операции):

	Название	Сигнатура	Назначение
2.4.1.	Get	<TElement> Get(int index)	Получение элемента по индексу.

2.4.2.	GetFirst	<TElement> GetFirst()	Получить первый элемент последовательности
2.4.3.	GetLast	<TElement> GetLast	Получить последний элемент последовательности
2.4.4.	GetSubsequence	Sequence<TElement> GetSubsequence(int startIndex, int endIndex)	Получить подпоследовательность: начиная с элемента с индексом startIndex и заканчивая элементом с индексом endIndex
2.4.5.	Append	void Append(<TElement> item)	Добавить элемент в конец последовательности
2.4.6.	Prepend	void Prepend(<TElement> item)	Добавить элемент в начало последовательности
2.4.7.	InsertAt	void InsertAt(int index, <TElement> item)	Вставить элемент в позицию index (существующие элементы сдвигаются вперед на 1).
2.4.8.	Remove	void Remove(<TElement> item)	Удаляет элемент из последовательности. Если элемента не было, ничего не происходит.

3. Требования к структурам данных и алгоритмам

3.1. Для хранения данных реализовать следующие варианты:

3.1.1. Использовать динамический массив

3.1.2. Использовать связные списки

Выполнить реализацию, использующую массивы (ArraySequence), и реализацию, использующую связные списки (ListSequence). Сделать оба эти класса наследниками абстрактного класса Sequence.

3.2. Все методы, принимающие на вход индексы, должны выбрасывать исключение в ситуациях, когда индекс выходит за допустимые границы: меньше 0 или больше или равен числу элементов в последовательности.

4. Требования к интерфейсу

Консольное приложение

5. Требования к системе сборки

Make/CMake

6. Требования к unit-тестам

6.1. Написать, по крайней мере, один тест, реализующий следующий сценарий:

- 6.1.1. Создать пустую последовательность (TElement = int).
- 6.1.2. Проверить, что длина = 0
- 6.1.3. Добавить (Append) элемент «23».
- 6.1.4. Проверить, что длина = 1
- 6.1.5. Проверить, что GetFisrt возвращает «23»
- 6.1.6. Проверить, что GetLast возвращает «23»
- 6.1.7. Проверить, что Get(0) возвращает «23», а Get(-1) и Get(1) – выбрасывают исключение.
- 6.1.8. Добавить (Append) элемент «43»
- 6.1.9. Проверить, что длина = 2
- 6.1.10. Проверить, что GetFisrt возвращает «23»
- 6.1.11. Проверить, что GetLast возвращает «43»
- 6.1.12. Проверить, что Get(0) возвращает «23», Get(1) – «43», а Get(-1) и Get(2) – выбрасывают исключение.
- 6.1.13. Добавить (Prepend) элемент «53»
- 6.1.14. Проверить, что длина = 3
- 6.1.15. Проверить, что GetFisrt возвращает «53»
- 6.1.16. Проверить, что GetLast возвращает «43»
- 6.1.17. Проверить, что Get(0) возвращает «53», Get(1) – «23», а Get(-1) и Get(3) – выбрасывают исключение.
- 6.1.18. Проверить, что GetSubsequence(1,1) возвращает последовательность, у которой:
 - 6.1.18.1. длина =1
 - 6.1.18.2. первый и последний элемент – 23

7. Требования к среде функционирования

Docker-контейнер с базовым образом Linux (любой дистрибутив). Реализовать Dockerfile выполняющий установку зависимостей, клонирование исходных текстов вашего приложения (из удаленного репозитория), запуск процесса сборки (в выбранной системе сборки).

8. Требование к дистрибутиву приложения

Дистрибутив составляют следующие файлы:

- исполняемый файл модульного тестирования;
- исполняемый файл приложения;
- файл разделяемой библиотеки (*.so).